

Workshop On Intrusion Detection Using ML Techniques

Milestone 3 Final Report: Pipeline Implementation, Evaluation & Intelligent Ensemble Optimization

Project & Metadata Overview

Table 1. Team members.

Full name	Student ID	Email
Nitsan Honen	207032764	nitsan.honen@mail.tau.ac.il
Itamar Ben Nun	322520255	itamarbennun@mail.tau.ac.il
Rotem Hayik	322958471	rotemhayik@mail.tau.ac.il

Table 2. Dataset and submission metadata.

Field	Project specification
Primary Dataset A	Mixed-source DNS collection: Domainator, GraphTunnel, DACA and CTU Normal, frozen Train, Validation and Test.
Generalization Dataset B	Frozen External Test: 77 attack recordings, including additional tools and scenarios, no benign traffic.
Code Repository & Branch	Repository URL: https://github.com/RotemHayik/Workshop-On-Intrusion-Detection-Using-ML-Techniques-.git Branch: main Package: 02_code
Model Artifact Archive	Drive Folder: https://drive.google.com/drive/folders/1XxVHtEaRGwXn-Uf_rFSsPzUN4lSoWF4O?usp=sharing Package: 05_models
Report / experiment dates	7 September 2026

Short result summary: We trained and evaluated four different detectors and found that Random Forest was the best single model on the internal Test set. We then combined the models in a cascade: this reduced missed attacks from four to one without adding false alarms on Dataset A. The LLM was tested only on a small number of difficult cases, and its mixed results are why the report does not claim that automatic LLM overrides are already safe for deployment.

Part 1: Modular Pipeline Architecture & Algorithmic Design

1.1 Pipeline Architecture

We organized the code as a pipeline so that each part has one clear job. First we convert the different DNS datasets into one common format, then we clean and scale the data, then create the three representations needed by the models, train the models, and finally evaluate them. The important design choice is that dataset-specific details are handled only at the beginning, after that, every model receives data in a consistent structure.

Table 3. Five-stage pipeline and stable interfaces.

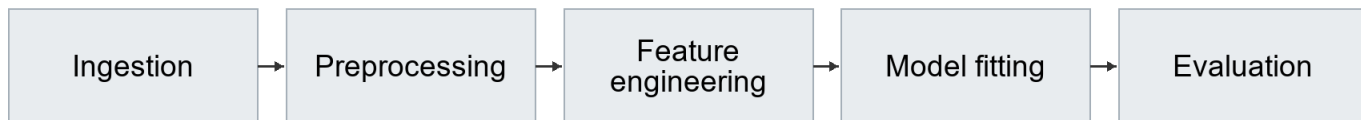
Stage	Implementation and output contract
1 Ingestion	dnsids.prepare maps source fields and validates canonical fields, labels and recording boundaries.
2 Preprocessing	The same cleaning, Train-fitted robust scaling and median imputation, fixed transforms are applied to Validation and both tests.
3 Feature engineering	Non-overlapping windows: up to 128 transactions, tails retained at 32 or more. Outputs: 42 tabular features, sequence arrays and within-window graphs.
4 Model fitting	dnsids.train consumes aligned representations, class/recording balancing is confined to Train. Validation selects models and thresholds.
5 Evaluation	dnsids.evaluate, diagnose and cascade produce frozen predictions, metrics, error evidence and local arbitration logs.

We separate information that is useful for prediction from information that is useful only for tracking where a sample came from. For example, a source name or recording ID could accidentally reveal the answer because some datasets contain mostly one class, so we do not let the models use those fields. We still keep those fields outside the model so we can split recordings safely, balance the data, and later investigate where errors came from.

Sequences contain 128×15 continuous channels plus query-type and response-code IDs and a validity mask. Graphs contain 13 node attributes and directed topology.

The actual Python implementation can be found here: [02 code/dnsids](#), and full pipeline diagram and supporting exploration figures here: [07 figures](#).

Figure 1. A visual overview of the complete workflow: from canonical input to frozen evaluation.



1.2 Environment, Tooling & Reproducibility Specifications

Table 4. Runtime environment, dependencies and deterministic settings

Item	Executed configuration
Hardware / OS	Windows 11 build 26200, Intel i5-10210U, 4 cores / 8 threads, 8.33 GB RAM, CPU-only computation.
Dependencies	Python 3.12.14, NumPy 2.2.6, pandas 2.2.3, SciPy 1.15.3, scikit-learn 1.7.1, PyTorch 2.8.0+cpu.
Seeds	42 for Python, NumPy, scikit-learn, PyTorch and the sampling generator. Neural repeats: 17 and 2026. Deterministic PyTorch algorithms enabled, no CUDA backend used.
Splits / resources	Fixed Part 2 recording manifest, no new random splitter. Three detector CPU threads, neural loaders use zero workers.

1.3 Class Imbalance Mitigation Strategy

Train has 5,530 benign and 5,441 malicious windows, so class totals are nearly balanced. The remaining imbalance is that some recordings contain many more windows than others. For each class, we assign equal total weight to each recording, then equal weight to its windows, weights are normalized to mean one.

RF uses these sample weights because it can directly receive a weight for every training row. Neural training draws 10,971 windows per epoch with replacement using the same weights and ordinary binary cross-entropy. IF fits benign windows only: 64 draws from each of 49 benign recordings, yielding 3,136 training draws.

Mathematically, a window in recording g of class y receives weight proportional to 1 divided by the product of the number of recordings in class y and the number of windows in g . Each class therefore has equal aggregate mass. Every sampled index belongs to Train, the intersection of recording IDs from any two splits is empty.

We chose weighting and balanced sampling instead of creating synthetic DNS examples. Methods such as SMOTE make new points by interpolating between existing examples, but that can create combinations that do not correspond to a real DNS sequence, query type, or graph. We therefore kept the real samples unchanged and changed only how strongly or how often they are used during training. All scaling and imputation parameters were also learned from Train only so information from Validation or Test could not leak into training.

1.4 Validation Strategy & Explicit Hyperparameter Optimization

We keep the Train, Validation, Test, and External Test recordings completely separated. Validation is the only held-out split we are allowed to use while making choices. For each model we choose a threshold that tries to catch as many attacks as possible while keeping the Validation false-positive rate at or below 1%. Once the model and threshold are selected, they are frozen before we look at Test performance.

Table 5. Locked model configurations and sensitivity dimensions

Model	Executed baseline and parameter changes
RF	300 trees, depth 12, minimum leaf 5, minimum split 2, sqrt features, Gini, bootstrap on, class_weight=None because sample weights are explicit, random_state=42, n_jobs=3. One-at-a-time variants: depth 20, leaf 10 or 500 trees.
IF	300 trees, max_samples=256, max_features=1.0, contamination=auto, bootstrap off, random_state=42, n_jobs=3. Variants: 1,024 samples, half of features or 500 trees. Final refinement: 1,024 samples and 29 retained features.
BiLSTM	22 input channels: 15 numeric + query-type embedding 4 + response-code embedding 3. One bidirectional layer, hidden size 32 per direction, output dropout 0.2, batch 64. Variants: hidden 64, two layers or learning rate 0.0003.
GraphSAGE	13 input attributes, two mean-neighbor layers of width 32, separate self/neighbor maps, ReLU and L2 normalization, mean graph pooling, dropout 0.2, batch 32. Variants: width 64, three layers or learning rate 0.0003.

Both neural models use AdamW optimizer and stopping rules: learning rate 0.001, weight decay 0.0001, betas (0.9, 0.999), epsilon 1e-8, gradient clipping 1.0, we allow training for at most 24 epochs, but stop earlier when Validation AUPRC stops improving for eight epochs. BiLSTM processes exact valid lengths to prevent padding from affecting backward states.

Depth/width regulate capacity, leaf size regularizes RF, tree count stabilizes ensembles, IF subsample size controls isolation granularity, feature fraction tests redundancy, learning rate controls optimization. All defaults and overrides are archived in 03_config and each run configuration.

Locked window thresholds: RF 0.850396, Isolation Forest (IF) base 0.705569 and final 0.736792, BiLSTM 0.983997, GraphSAGE 0.795319. A score at or above its threshold is malicious, IF uses negative score_samples, so larger values indicate greater anomaly.

1.5 Hyperparameter Sensitivity & Stability Analysis

Table 6. Validation sensitivity at each configuration's own frozen threshold

Model	Baseline / three controlled parameter changes	FP counts	F1 range %
RF	Base, depth 20, leaf 10, trees 500	0 / 0 / 0 / 0	100-100
IF	Base, samples 1,024, features 0.5, trees 500	7 / 3 / 7 / 7	99.703-99.872
BiLSTM	Base, width 64, lr 0.0003, layers 2	1 / 5 / 1 / 4	99.787-99.957
GraphSAGE	Base, width 64, lr 0.0003, layers 3	0 / 0 / 0 / 0	100-100

All 16 configurations achieved 100% Validation recall. Wider/deeper BiLSTMs increased false positives, reducing their learning rate tied the baseline. RF and GraphSAGE had flat operating-metric responses across the tested settings. Increasing IF subsample size reduced false positives. The final IF feature refinements are isolated in Section 2.2.

Table 7. Neural baseline stability across fixed seeds

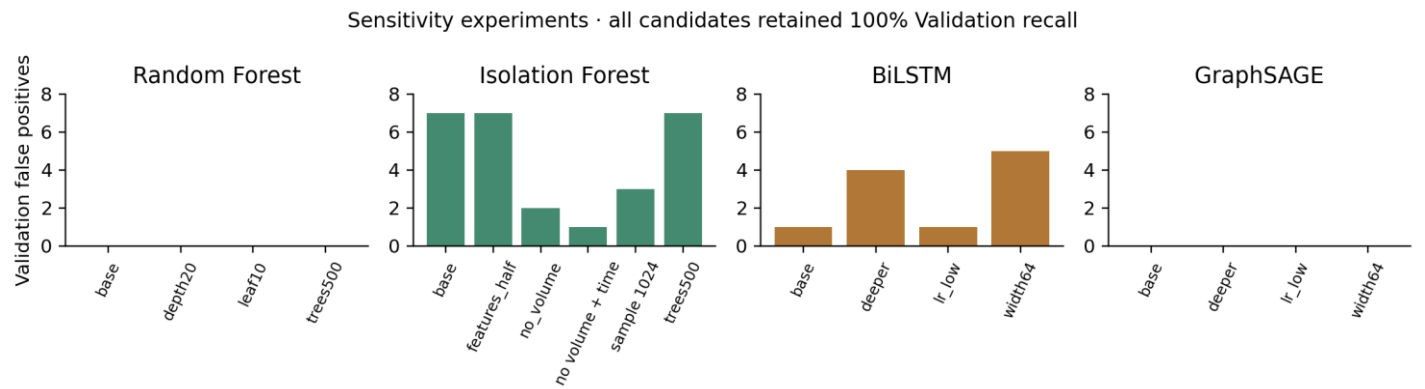
Model	FP at seeds 42 / 17 / 2026	Validation F1 %	Threshold range
BiLSTM	1 / 2 / 2	99.915-99.957	0.509543-0.983997
GraphSAGE	0 / 0 / 0	100-100	0.786035-0.798748

We re-ran the selected neural architectures with three random seeds. GraphSAGE gave the same perfect Validation classifications each time. BiLSTM stayed very accurate but its chosen threshold changed considerably, which tells us its score scale is sensitive to initialization even when the final decisions remain similar. We repeated the two neural baselines with different random seeds to check whether the result depended on one lucky initialization. Their final classification quality stayed high, but the exact BiLSTM score threshold moved a lot between seeds. That tells us the binary decisions are more stable than the raw score

scale. We kept seed 42 for Test because it was chosen in advance, not because it happened to be the best seed.

This is a bounded one-at-a-time study, not exhaustive grid search or cross-validation. We ran a small controlled set of changes that was realistic on the available computer. The complete learning curves and timings are saved in: [06 results/runs](#), [validation_sensitivity.csv](#), [seed_stability.csv](#) and [07 figures](#).

Figure 2. Validation false positives across 16 hyperparameter configurations and two additional Isolation Forest feature refinements. All 18 configurations achieved 100% malicious recall at their respective Validation-selected thresholds. Additional neural seed repeats are not shown.



Part 2: Comparative Performance, Forensic Error Analysis & Generalization

2.1 Sample-Level Forensic Error Analysis (per dataset)

Table 8. Frozen evaluation populations after window construction

Partition	Windows	Benign / malicious	Recordings
Train	10,971	5,530 / 5,441	130
Validation	2,350	1,177 / 1,173	25
A - Test	2,339	1,179 / 1,160	26
B - External Test	7,547	0 / 7,547	77

Table 9. Dataset A Test confusion counts and window metrics, all rates are percentages

Model	FP	FN	Acc.	Prec.	Recall	F1	FPR	FNR
RF	0	4	99.83	100.00	99.66	99.83	0.000	0.345
BiLSTM	0	13	99.44	100.00	98.88	99.44	0.000	1.121
GraphSAGE	8	83	96.11	99.26	92.84	95.95	0.679	7.155
IF base	10	2	99.49	99.14	99.83	99.48	0.848	0.172
IF final	5	7	99.49	99.57	99.40	99.48	0.424	0.603

Dataset A Test AUROC values were 1.0000 for RF, 1.0000 for BiLSTM, 0.9993 for GraphSAGE, 0.9992 for baseline IF, and 0.9999 for refined IF, rounded to four decimal places. These ranking metrics are distinct from performance at the frozen operating thresholds.

Dataset A contains both benign and malicious windows, so we can calculate the full confusion matrix and metrics such as F1, false-positive rate, and AUROC. It also reminds the reader that the final Isolation Forest is the 29-feature version selected during Validation, not the original 42-feature baseline.

Table 10. Dataset B External Test attack detection, 7,547 malicious windows

Model	TP	FN	Recall %	FNR %
RF	7538	9	99.881	0.119
BiLSTM	7479	68	99.099	0.901
GraphSAGE	7515	32	99.576	0.424
IF base	7530	17	99.775	0.225
IF final	7485	62	99.178	0.822

B has no benign samples: FP/FPR cannot be evaluated, and binary accuracy, precision, F1 and AUROC are not reported. All errors below are evaluated at the locked Validation thresholds, not at 0.5.

Table 11. Error characteristics for every detector on both held-out datasets

Model	A - Test failure pattern	B - External failure pattern
RF	4 FN in one Domainator Saitama handshake recording, no FP. Example: repeat ratio 0.9766, score 0.8153 below 0.8504.	8 FN from Domainator download traffic and 1 GraphTunnel wildcard window. Repetitive payload and near-threshold scores recur.
BiLSTM	9 FN in one GraphTunnel iodine recording, 3 DACA FN and 1 Domainator FN, no FP. High-ranking scores still fell below the strict threshold.	33 Domainator FN in 5 recordings and 35 GraphTunnel FN in 5 recordings, download and additional tool sequences differ from Validation.
GraphSAGE	79 iodine FN in one GraphTunnel recording and 4 DACA FN. All 8 FP belong to one CTU benign recording.	22 Domainator FN in 5 recordings and 10 GraphTunnel FN in 6 recordings, including wildcard traffic. Proxy topology is not a full recursive-resolution graph.
IF	Base: 10 FP (9 CTU, 1 GraphTunnel), 2 Domainator FN. Final: 5 FP (4 CTU, 1 GraphTunnel), 7 FN (6 Domainator, 1 DACA).	Base: 17 Domainator FN. Final: 46 Domainator and 16 GraphTunnel FN, removing size/timing cues sacrifices attack evidence.

Validation exposed the refinement target: all seven IF baseline false positives came from one CTU recording, with byte features above benign Train maxima. BiLSTM had one CTU false positive, RF and GraphSAGE had no Validation errors.

These are feature-supported interpretations, not proofs of causal model reasoning. All FP/FN rows and 104 Test / 117 external disagreements are retained across the four models plus IF refinement. Twenty distinct raw windows covering 26 model/source/error categories matched all 52 audit features within the recorded numerical tolerances.

Exact results can be found here: [06 results/error analysis](#) and [representative error cases.csv](#), raw records and feature reconstruction checks: [04 data/raw examples](#).

2.2 Iterative Pipeline Optimization & Verification

Table 12. Validation changes to IF, with thresholds selected independently at each step

Change	Features	FP	FN	F1 %
Baseline	42	7	0	99.703
Increase subsample	42	3	0	99.872
Remove volume	37	2	0	99.915
Also remove timing	29	1	0	99.957

First we increased the Isolation Forest subsample size because that already reduced Validation false positives. Next we removed five byte/payload features that appeared suspiciously tied to the CTU acquisition pattern, and then removed eight timing features as a second hypothesis. We did not change the scaler or the frozen split during these experiments, so the comparison isolates the effect of these specific decisions.

Table 13. Held-out consequences of the complete IF refinement

Metric	IF baseline	IF final
A: FP / FN	10 / 2	5 / 7
A: FPR / FNR %	0.848 / 0.172	0.424 / 0.603
A: F1 %	99.485	99.482
B: FN / recall %	17 / 99.775	62 / 99.178

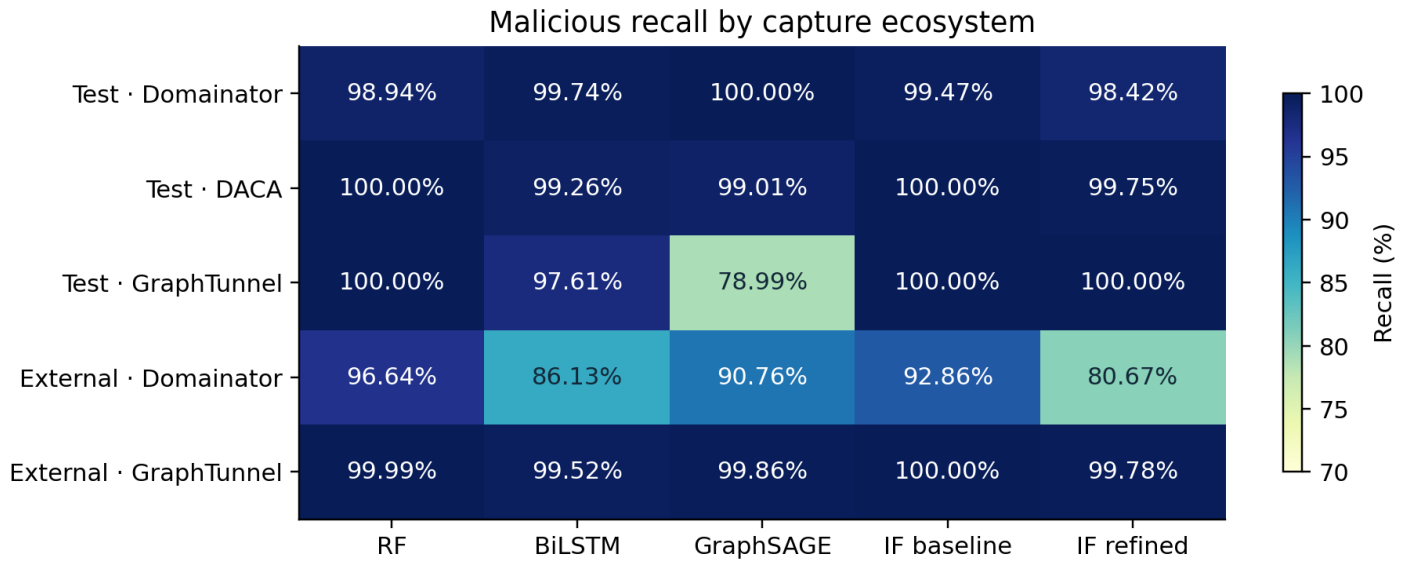
The final refinement clearly reduced false alarms on Dataset A, but it also missed more attacks. This is an important negative result: a change can look better on Validation for the intended objective and still have a trade-off on held-out data. We report that trade-off instead of retuning after Test, because changing the model after seeing Test results would make Test part of the tuning process.

2.3 Cross-Dataset Variance & Environmental Distribution Shift

A and B differ in tools, scenarios and recording mixture, not in every capture ecosystem. RF recall was 99.655% on A versus 99.881% on B, BiLSTM 98.879% versus 99.099%, GraphSAGE 92.845% versus 99.576%, final IF 99.397% versus 99.178%. GraphSAGE's apparent gain is largely a mixture effect: 79 A misses cluster in one iodine recording. B does not establish generalization to unfamiliar benign traffic.

Most benign training traffic comes from GraphTunnel, while CTU contributes very little benign Train data. Even though recordings do not overlap between splits, a model may still learn characteristics associated with a source instead of only learning universal benign-versus-malicious behavior. We added CTU normal traffic to reduce this shortcut, but the imbalance between ecosystems is still present and must be stated openly.

Figure 3. Malicious-window recall (%) by capture source on Dataset A Test and Dataset B External Test, using frozen Validation thresholds. Rows contain 379, 405, 376, 238 and 7,309 malicious windows, respectively. IF refined denotes the final 29-feature model. These values do not measure benign-traffic performance.



2.4 Empirical Benchmarking Against Literature (Step 2 Alignment)

Table 14. Published results and corresponding project results

Source / family	Published result	Our result and comparison boundary
Domainator RF	F1 96.6%, macro precision 95.8%, macro recall 97.0%, FPR 1.2%.	A: F1 99.827%, recall 99.655%, FPR 0%. Different windowing, source mixture and class averaging prevent direct ranking.
Sakarkar et al. BiLSTM anchor	80:20 LSTM experiment: precision 99%, recall 98%, F1 98%.	A: precision 100%, recall 98.879%, F1 99.436%. Their embedded packet text differs from our numeric transactions and DNS categories.
GraphTunnel GraphSAGE	Wildcard DNS F1 99.78%, 100% accuracy in the stated non-wildcard detection setting.	A: F1 95.947%, recall 92.845%. Our proxy lacks full recursive-resolution paths and does not use stored edge attributes.
KRTunnel Isolation Forest	Reported accuracy 98.1% for unseen DNS tunnels on Android.	A: baseline and final accuracy both 99.487%. Different mobile data and inputs, equal accuracy hides the FP/FN tradeoff.
Mitsuhashi et al. Flow-based trees	CIRA-CIC-DoHBrw-2020: 99.81% DoH filtering, 99.99% suspicious-DoH detection, 97.22% tool identification accuracy.	Step 2 supplied size/timing feature principles. We classify ordinary DNS windows, not encrypted DoH flows or tool identity, no equivalent three-stage task was measured.

Domainator, for example, discusses repetitive communication patterns that resemble some of our missed windows. GraphTunnel also uses richer recursive graph information than our proxy graph, which is a plausible reason its published graph results are stronger. These are careful comparisons, not one-to-one replications.

Part 3: Advanced Hybrid Pipeline

3.1 Multi-Stage/Model Cascading Architecture

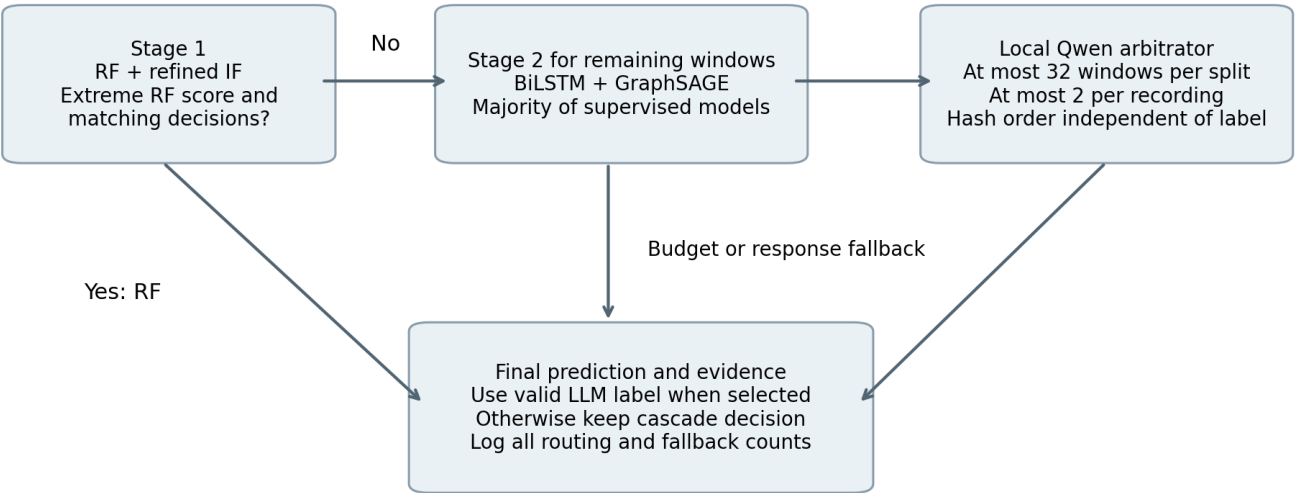
The cascade was designed after looking at Validation behavior, before evaluating held-out Test data. Random Forest is the first supervised detector because it is both fast and very accurate. Isolation Forest is used as a second opinion: when it disagrees with RF, or RF is not very confident according to our fixed score bands, we

send the window to the more expensive neural models. This lets the models complement one another instead of simply averaging everything.

Table 15. Executed behavioral routing rules

Stage	Rule
1 - RF + final IF	If the frozen binary decisions agree and the RF score is ≤ 0.2 or ≥ 0.8 , accept RF. These score bands are heuristics, not calibrated confidence.
2 - Supervised vote	Otherwise use the majority of RF, BiLSTM and GraphSAGE decisions, each at its own frozen threshold. Every such window is eligible for LLM arbitration.
3 - Local arbitration	Select by ascending SHA256(sample_id), capped at 2 windows per recording and 32 per split. Valid responses may override under the Validation gate, budget/response failures retain the vote.

Figure 4. Behavioral cascade with local LLM arbitration. RF and refined IF screen each window, remaining cases use an RF/BiLSTM/GraphSAGE majority vote, followed by budgeted Qwen arbitration where selected. Budget exclusions or response failures retain the vote. The routing and override policy was frozen on Validation.



Validation froze the override rule before Test; held-out harm is reported without retuning.

3.2 Multi-Stage/Model Cascading Architecture Accuracy

Table 16. RF versus the cascade before LLM arbitration

Data	System	F1 %	FPR %	FNR %	FN
A	RF	99.827	0.000	0.345	4
A	Cascade	99.957	0.000	0.086	1
B	RF	N/A	N/A	0.119	9
B	Cascade	N/A	N/A	0.212	16

On Dataset A the cascade did what we wanted: it changed three of the four RF misses into correct detections and did not create any new false positives. However, on Dataset B the standalone RF actually missed fewer attacks than the cascade. Therefore, the cascade is not uniformly better.

Disclaimer: For this experiment we already had prediction scores from all four models and simulated the routing decisions on those saved outputs. That is enough to measure classification accuracy of the cascade, but it does not prove that a live system would actually save the same amount of computation by skipping neural models on easy cases. Also, the cascade's final output is mostly a yes/no value, so its AUROC is not directly comparable with a detector that produces a continuous ranking score.

Part 4: Local LLM Configuration, Prompt Design & Evaluation

4.1 Local Model Configuration & Runtime Constraints

Table 17. Local model, inference settings and resource limits

Setting	Executed value and rationale
Model and license	Qwen2.5-1.5B-Instruct, official GGUF repository, Apache 2.0. Small instruction-tuned model chosen for structured local triage on the CPU laptop.
Weights	Q4_K_M, 1,117,320,736 bytes (1.117 GB). Repository revision 91cad51170dc346986eccefdc2dd33a9da36ead9.
Backend	llama.cpp b10819, Windows x64 CPU build, 127.0.0.1:8765, one slot, two CPU threads. No cloud API.
Context and memory	Context: 2,048 tokens. A follow-up memory measurement on 12 September 2026, performed on the Intel Core i5-10210U, recorded a peak resident set size of 1,338.7 MiB (1,403.8 MB), measured using Windows PeakWorkingSet64 for the local llama-server process during model loading and one saved Test request. The run used two CPU threads and max_tokens=120. No GPU VRAM was used.
Determinism	temperature=0; seed=42. top_p was not explicitly supplied. The same inference settings were used across all conditions in the follow-up prompt-content ablation.
Output and timeout	max_tokens=120 per request, JSON response format, 90-second request timeout. In the original 21-request arbitration experiment, all responses were valid JSON and the longest completion contained 113 tokens. The separate prompt-content ablation is reported in Section 4.2. Launcher default -n 160 is superseded by the request limit.
Fallback	Invalid JSON, missing evidence/limitation, unsupported label or request failure preserves the cascade decision. The same applies to unselected candidates.

The LLM only needs to return a short structured answer: a label, one evidence statement, and one limitation statement. We therefore limited the maximum output length and used deterministic decoding settings so repeated runs would be as consistent as possible. Deterministic generation improves reproducibility, but it does not make the LLM's judgment automatically correct.

We did not fine-tune Qwen on our dataset and did not give it internet access or external tools. It only sees the numeric context supplied in the prompt. Because local CPU inference is slow, we limit how many windows can reach the LLM. Importantly, windows that are not selected are still included in final performance calculations using the cascade decision, so the reported population metrics do not ignore the difficult cases that exceeded the LLM budget.

4.2 System Prompt & Telemetry Injection Template

Listing 1. Exact system prompt sent in every measured request.

You assess DNS tunneling from a completed traffic window. Use only the supplied numeric observations, Train reference medians and detector decisions. Anomaly alone is not proof of attack: legitimate background traffic can have large responses. Combine lexical payload, domain concentration, recurrence and query-type evidence. No labels, source identities or tool identities are provided. Return a JSON object with label (benign or malicious), evidence (one short sentence naming the observed features), and limitation (one short uncertainty statement). Do not invent packet content or reveal a hidden reasoning trace.

This is the exact instruction given to Qwen for every evaluated request. We tell it that one unusual value is not enough to prove an attack and that it should compare several kinds of evidence, such as domain concentration, repetition, entropy, and query types. We also force a small JSON output so the answer can be parsed automatically and checked for validity.

Listing 2. User-message template serialized as JSON, without a text prefix.

```
{
  "window_features": <F>,
  "train_recording_median_reference": {
    "benign": <B>, "malicious": <M>
  },
  "detector_decisions": {
    "rf": "<R>", "if": "<I>",
    "bilstm": "<L>", "sage": "<G>"
  },
  "rf_score": <S>
}
```

F, B and M are JSON objects with the ten exact feature keys shown in Table 18. `F` is the current window, `B` and `M` are typical benign and malicious Train references, and `R/I/L/G` are the four frozen model decisions. The raw Random Forest score provides an additional numeric model signal alongside the frozen binary decisions. The operating threshold is not included in the prompt, so the LLM is not given an explicit distance to that threshold.

Finite numbers are rounded to four decimals, non-finite window values become JSON null. References are computed by taking each recording's feature median, then the median across recordings within each Train class. The generated message is `json.dumps(prompt_for(row, refs))`, there is no additional user instruction.

No true window label, raw domain, source/tool identity, endpoint or recording identifier enters either message. The two reference classes describe Train context rather than revealing the current window's label. Sample IDs are retained only in the local audit log.

The following is the full numeric context for Test window test-3f672216ed3a6d80-000001, an audited benign CTU case. Its identifier and ground truth are shown here for forensic traceability and were not sent to Qwen.

Table 18. Exact feature objects F, B and M for the audited request

Feature key	F: window	B: benign	M: malicious
query_name_len_mean	25.7266	14.2344	41.6250
subdomain_len_mean	9.3906	0.9531	27.1484
query_entropy_mean	3.5593	3.2599	3.7816
longest_subdomain_entropy_mean	1.3315	0.2485	2.8934
transaction_bytes_mean	229.4688	173.8906	390.3906
top_base_domain_share	0.0625	0.0234	1.0000
unique_subdomain_ratio	0.9453	0.1094	0.5078
repeated_query_ratio	0.0547	0.0078	0.4922
qtype_a_ratio	0.0000	1.0000	0.0000
qtype_txt_ratio	0.0000	0.0000	0.0000

The remaining context was detector_decisions = {rf: benign, if: benign, bilstm: benign, sage: malicious} and rf_score = 0.5874. The numeric JSON objects above, combined with these fields and Listing 2, reconstruct the complete user message.

The ten features were chosen to give the LLM several different views of behavior instead of only one suspicious number. Some features describe string length and randomness, some describe how concentrated or repetitive the DNS activity is, and others describe DNS query types. The benign and malicious references help the LLM understand whether a value is closer to what we normally saw in each Train class.

In this benign Test case, Qwen returned a malicious label, and its generated evidence emphasized query-name length. Query-name length appeared in all 21 explanations from the original arbitration experiment. This recurring explanation pattern does not establish the causal importance of that feature or demonstrate faithful model reasoning. A separate follow-up prompt-content ablation evaluated all seven escalation-eligible Validation windows under three conditions: the original prompt, removal of train_recording_median_reference, and removal of both detector_decisions and rf_score. The system prompt, model, and inference settings were held fixed, including temperature=0, seed=42 and max_tokens=120. Across 21 additional requests, the original prompt produced seven valid responses with six correct labels. Removing the reference medians produced two valid responses, one correct, and five

incomplete JSON responses. Removing detector context produced seven valid and correct responses, changing one label relative to the original prompt. These findings demonstrate sensitivity of both classification and output validity to the supplied context. However, all seven windows were benign and came from one CTU recording, so the results do not establish general superiority of any prompt variant. This diagnostic follow-up did not change the original prompt, arbitration policy, or held-out results.

4.3 LLM Arbitration Performance

Table 19. All escalated windows and the complete bounded arbitration sample

Split	Eligible	LLM	Deferred	Correct before / after	Fixed / harmed
Validation	7	2	5	2 / 2	0 / 0
A - Test	18	6	12	6 / 5	0 / 1
B - External	61	13	48	9 / 12	3 / 0

In the original arbitration experiment, only the windows selected by the frozen budget were sent to Qwen: 21 requests in total. The cascade correctly classified 17 of these windows before arbitration and 19 afterward. However, this small, non-random operational sample does not support a claim of general LLM accuracy of 19/21 across DNS traffic.

Table 20. Population impact with deferred windows retained in their cascade decision

Data	System	FP / FN	F1 %	FPR %	FNR %
A	Vote	0 / 1	99.957	0.000	0.086
A	Vote + LLM	1 / 1	99.914	0.085	0.086
B	Vote	N/A / 16	N/A	N/A	0.212
B	Vote + LLM	N/A / 13	N/A	N/A	0.172

We decided in advance that LLM overrides would be allowed only if they did not make Validation worse and kept the Validation false-positive rate within the limit. The gate passed, but it was based on only two Validation requests from one recording. We therefore kept the policy frozen for Test and reported what happened: one harmful override on A and three useful corrections on B. We did not change the rule after seeing those results.

Mean observed request times were 41.1 seconds on Validation, 35.9 on A and 22.5 on B, some calls overlapped neural training. Local LLM calls were slow because everything ran on CPU, and some requests happened while neural models were also training. The timing numbers therefore describe this laptop experiment rather than a clean speed benchmark. We also checked the explanations themselves and found cases where the written evidence did not faithfully reflect all the supplied features. A response can be valid JSON and still contain weak reasoning.

The safest use of the LLM at this stage is as an analyst-assistance layer, not as an automatic final authority. An analyst can read its evidence together with the model outputs, but automatic overrides should be validated on a much larger and more diverse set containing both benign and malicious cases. We keep the negative Test result in the report to show that we did not hide unfavorable outcomes.

References and Reproducibility Evidence

- Petrov et al. (2025). Domainator: Detecting and Identifying DNS-Tunneling Malware Using Metadata Sequences. Section 4.1. Primary paper
- Sakarkar et al. (2021). Advance Approach for Detection of DNS Tunneling Attack from Network Packets Using Deep Learning Algorithms. ADCAIJ 10(3), 241-266. DOI
- Gao et al. (2024). GraphTunnel: Robust DNS Tunnel Detection Based on DNS Recursive Resolution Graph. IEEE TIFS 19, 7705-7719. Author paper
- Wang et al. (2022). KRTunnel: DNS Channel Detector for Mobile Devices. Computers & Security 120, 102818. DOI
- Mitsuhashi et al. (2021). Identifying Malicious DNS Tunnel Tools from DoH Traffic Using Hierarchical Machine Learning Classification. ISC 2021, 238-256. DOI
- Qwen team. Qwen2.5-1.5B-Instruct-GGUF. Official model repository, pinned revision in Section 4.1. Model and license
- ggml-org. llama.cpp, release b10819. Local CPU runtime and license. Pinned runtime
- Hamilton, Ying and Leskovec (2017). Inductive Representation Learning on Large Graphs. NeurIPS. GraphSAGE method paper

Reproduction and uncertainty

The repository contains prepared inputs, 22 completed training runs, frozen model configurations and checkpoints, and the prompts and responses from the original LLM arbitration experiment. The Qwen GGUF weights are distributed separately through the linked Model Artifact Archive. Checkpoint replay preserved every classification decision across ten Test and External Test prediction files; the maximum absolute score difference was 4.44×10^{-16} . Read [08 documentation/REPRODUCE.md](#) and [09 reproducibility](#) for commands and evidence.

Uncertainty uses 500 stratified recording-cluster bootstrap samples and exact recording-level binomial intervals. Only 11 benign Test recordings were represented, 0 false-positive recordings has a two-sided 95% upper bound of 28.49%. Large window counts do not remove this limitation. Extra figures are in [07 figures](#) and the complete source/tool matrix is in [performance by source and tool.csv](#).